

Bigdata 101

Hadoop Distributed File System - HDFS

Veysel Yüksel



Table of Contents

01

Hadoop Ekosistemi

02

HDFS Nedir

03

HDFS Mimarisi, Node Roller

04

Metadata ve Fsimage,Editlog

05

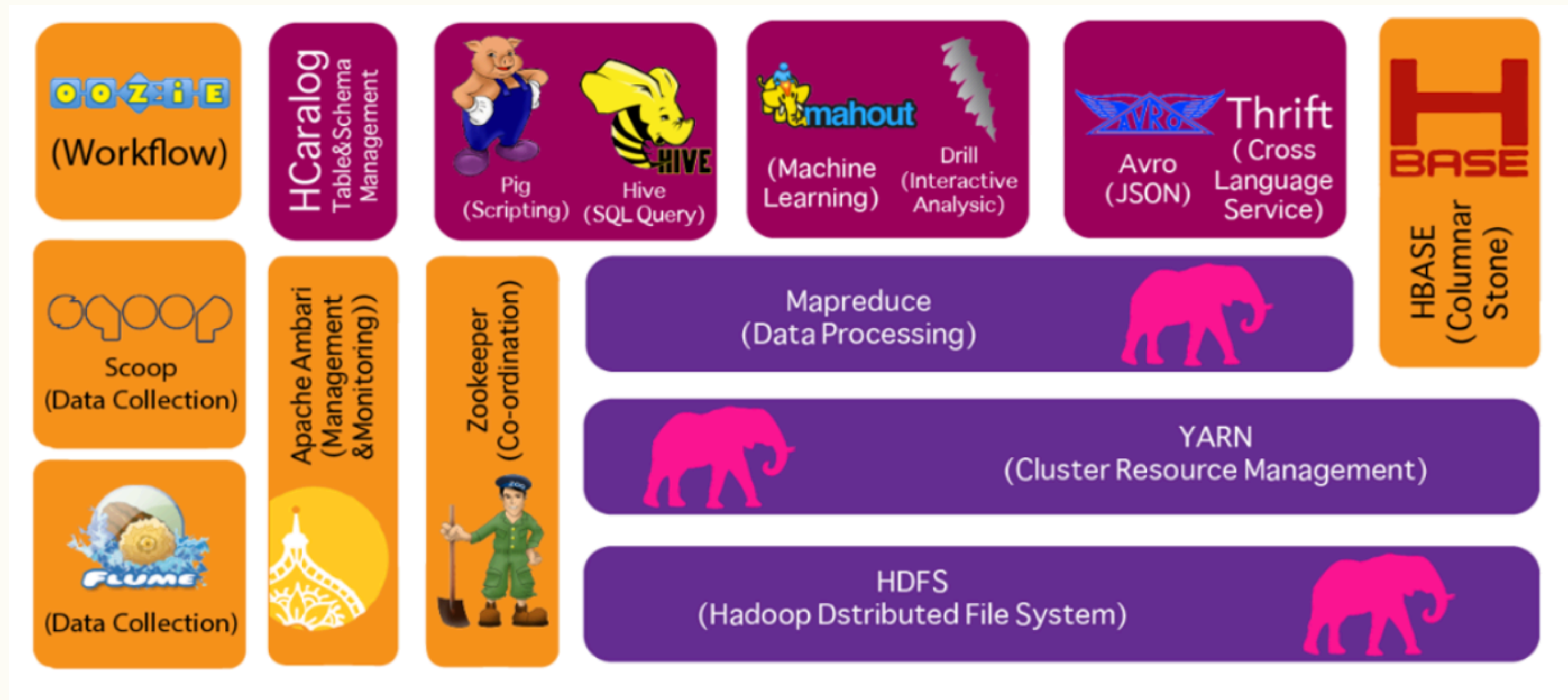
CheckPoint İşlemi

06

Read & Write Operations

Hadoop Ekosistemi

Hadoop büyük verinin depolanması ve işlenmesini sağlayan, birden çok servisi bünyesinde barındıran, apache tarafından geliştirilmiş open-source bir platformdur.



DAĞITIK DEPOLAMA (HDFS)



Verilen birçok sunucuya bölerek dağıtılır ve kopyalar. Bu şekilde büyük veriler güvenli bir şekilde saklanabilir.



DAĞITIK İŞLEME (MAPREDUCE)

Veriler, paralel olarak birçok sunucuda işlenir. Bu, büyük veri üzerinde hızlı analiz yapılmasını sağlar.



HATA TOLERANSI (FAULT TOLERANCE)

Veriler birden fazla kopya halinde saklandığı için bir düğüm çökerse bile veri kaybı yaşanmaz.

HADOOP'UN TEMEL ÖZELLİKLERİ



VERİ YERİNDE İŞLEME (DATA LOCALITY)

Veri neredeyse işlemin de orada yapılır, bu da ağ trafiğini ve gecikmeleri azaltır.



YÜKSEK ÖLÇEKLENEBİLİ- RİLİK (SCALABILITY)

Yatayda yeni makineler eklenerek sistem kapasitesi kolayca artırılabilir.



DÜŞÜK MALİYETLİ DONANIM KULLANIMI

Ucuz, sıradan donanımlar (commodity hardware) üzerinde çalışacak şekilde tasarlanmıştır.



AÇIK KAYNAK VE TOPLULUK DESTEĞİ

Apache tarafından geliştirilen açık kaynaklı bir projedir, geniş bir geliştirici topluluğu vardır.

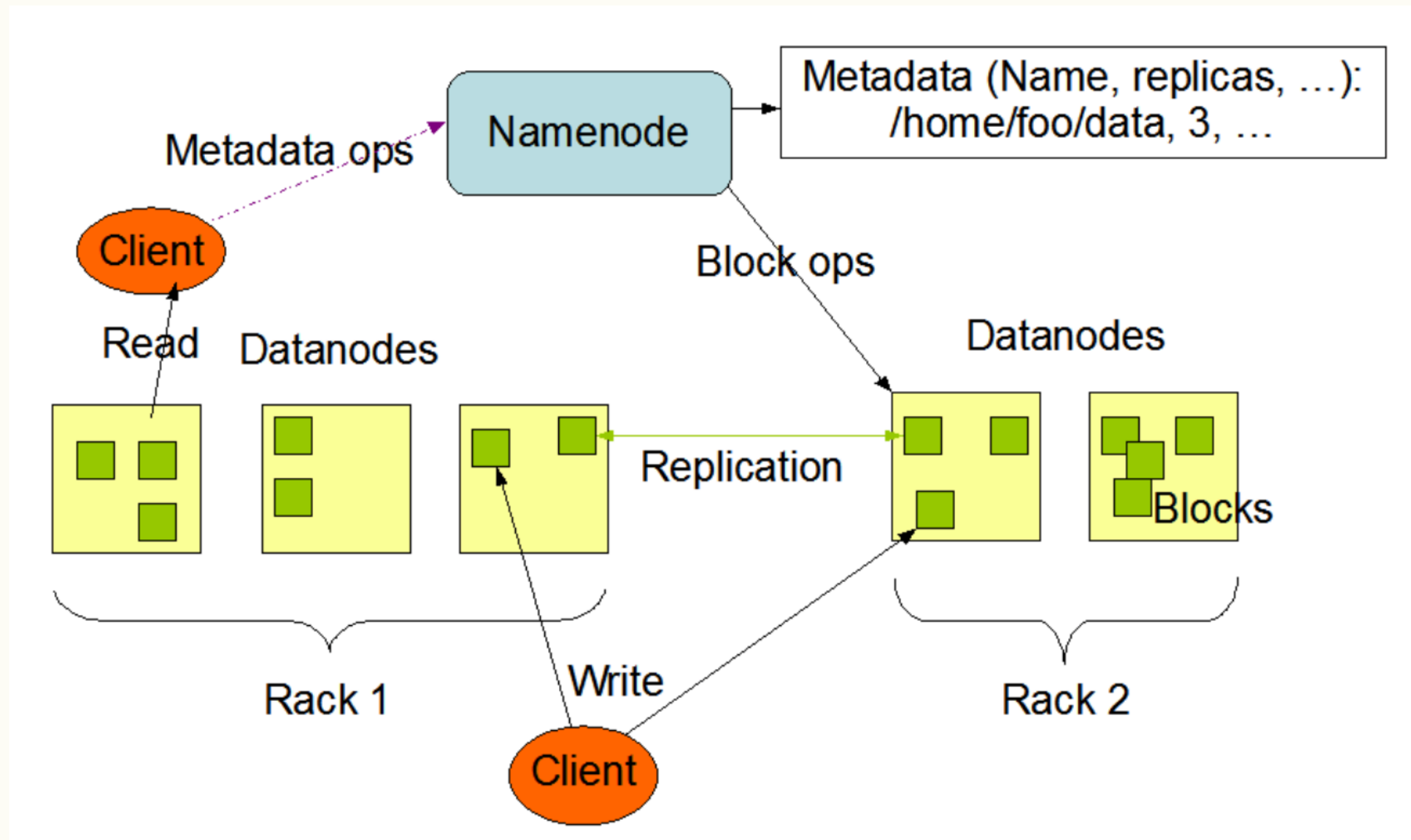
Hadoop Distributed File System - HDFS

Hadoop ekosisteminde, verinin depolanmasını sağlayan, dağıtık mimaride çalışan dosya sistemidir. Java ile geliştirilmiştir. HDFS dosyaları bloklara bölerek saklar ve her blok farklı sunucular üzerine replike edilerek high availability sağlanmış olur.

- Hata durumlarına karşı dayanıklı
- Dağıtık mimari, düşük maliyet
- Petabyte boyutlarında datayı bile sorunsuzca saklar.
- Default block size'ı 128/256 MB olabilir.
- Dosyalar WORM, write once read many, modeline göre depolandığından var olan dosyaları güncellemek yerine yeni versiyonunu yazma modeli ile çalışır.
- Büyük boyutlu dosyaların saklanması için geliştirildiğinden, küçük boyutlu çok fazla dosyanın saklanması performans sorunları yaşanmasına sebep olabilir.

HDFS

HDFS'de node'ların namenode, datanode, journalnode ve secondaryname node gibi farklı rolleri bulunabilir. Temelde HA özelliğinin aktif edilmediği bir HDFS cluster mimarisi şu şekildedir;



HDFS - Node Roles, Namenode

- HDFS, **master/slave** mimaride çalışır. Bir HDFS clusterda, filesystem namespace'i yöneten ve client'ların dosyalara erişimini düzenleyen **tek bir NameNode bulunur**. Bir clusterda **birden fazla DataNode bulunur**.

NameNode

- HDFS metadatasını tutar ve datanode'ların yöneticisi gibi düşünülebilir.
- Kullanıcılardan gelen okuma yazma işlemlerinin organizasyonunu sağlar.
- **Veri dosyalarının kendisi değil**, sadece bu verilerin **nerede bulunduğu** (blokların konumu) ve **nasıl organize edildiği** bilgisi burada tutulur.
- HDFS üzerinde namespace operasyonları adı verilen bir dizin veya dosya açma kapatma, yeniden isimlendirme gibi operasyonlar namenode tarafından gerçekleştirilir

HDFS - Metadata

- HDFS'te metadata, verinin nasıl organize edildiğinden, nerede saklandığından ve hangi kurallara göre erişileceğinden sorumlu yönetimsel bilgi/kayıttır.

- Dosya Sistemi Yapısı (Filesystem Namespace)
- Dosya Bilgileri
- Blok Bilgileri
- DataNode Bilgileri (Cluster Health Metadata)
- Snapshot Metadata
- Quota Bilgileri

HDFS - Metadata

Dosya Bilgileri

Özellik	Açıklama
Dosya Adı	/data/log.txt gibi tam yol.
Sahiplik Bilgileri	Kullanıcı adı (user) ve grup (group) bilgisi.
İzinler (Permissions)	Okuma, yazma, çalıştırma yetkileri (örnek: rw-r--r--).
Zaman Damgaları (Timestamps)	Oluşturulma, değiştirilme ve erişim zamanları.
Blok Sayısı	Dosya kaç bloktan oluşuyor (örn. 2 blok).
Blok Boyutu	Varsayılan olarak 128 MB (özelleştirilebilir).
Replication Factor	Her blok kaç kopya ile saklanacak (örnek: 3).
Snapshot bilgisi	Dosya snapshot kapsamında mı? (Yedeklenmiş hali var mı?).

Blok Bilgileri

Bilgi	Açıklama
Blok ID 	Her blok için benzersiz bir kimlik numarası (örnek: blk_1073741836).
Blok Boyutu	Gerçek boyutu (örneğin son blok genelde tam dolmaz).
Blok CRC	Blok veri bütünlüğünü doğrulamak için kullanılan kontrol kodu (checksum).
Blok Replikaları	Hangi DataNode'larda saklandığı bilgisi.
Blok Durumu	Sağlıklı mı, eksik mi, replike mi?

HDFS - Metadata

Datanode Bilgileri

Bilgi	Açıklama
Node ID / IP	DataNode'un adresi ve portu.
Blok Listesi	O DataNode üzerinde hangi bloklar mevcut.
Heartbeat Zamanı	En son ne zaman sinyal gönderdi?
Disk Kapasitesi	Toplam, kullanılabilir ve kullanılan disk alanı.
Rack Bilgisi	DataNode'un fiziksel ağı/rack'teki konumu.

Metada'nın Saklandığı Dosyalar

Dosya	Görev
fsimage	Tüm dosya sistemi yapısının snapshot'ı. RAM'e yüklenir.
edits	fsimage sonrası yapılan işlemlerin günlüğü (append log).
seen_txid	NameNode'un hangi işlem numarasına kadar işlediğini belirtir.
VERSION	Dosya formatı ve sürüm bilgisi içerir.

HDFS - Metadata

Metadata Neden Önemlidir?

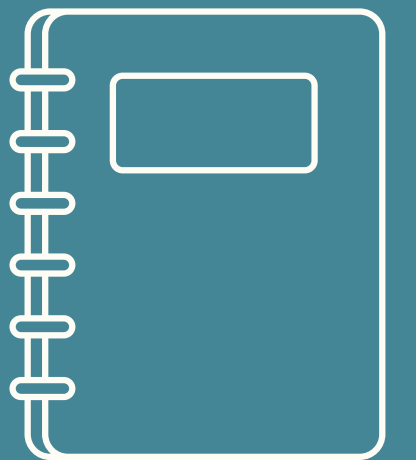
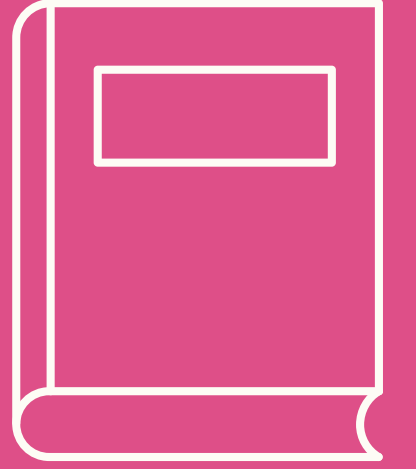
- **Veri Konumlandırma:** NameNode bu bilgilerle hangi dosya nerede bilir.
- **Veri Tutarlılığı:** Bozulmuş/veri kaybı durumunda yeniden kopyalama başlatılabilir.
- **Yük Dengeleme:** Blok dağılımı izlenerek eşitlenebilir.
- **Erişim Kontrolü:** Kullanıcı bazlı okuma/yazma kısıtları uygulanabilir.

HDFS-NameNode'un Görevleri

Görev	Açıklama
1. Namespace Yönetimi	Dosya/klasör yapısını yönetir (klasörler, izinler, sahiplik, zaman damgaları).
2. Blok Eşleme (Block Mapping)	Her dosya için: kaç bloktan oluşur ve bu bloklar hangi DataNode'larda yer alır bilgisi tutulur.
3. Replikasyon Kontrolü	Her veri bloğunun istenen replikasyon faktöründe saklanmasını sağlar (örneğin 3 kopya).
4. DataNode Sağlık Takibi	DataNode'lardan gelen heartbeat sinyallerini izler. Bir DataNode uzun süre cevap vermezse "dead" olarak işaretlenir.
5. Blok Bütünlüğü ve Yeniden Dağıtım	Eksik, bozuk veya fazla kopyalanmış blokları tespit eder ve blokların yeniden çoğaltılmasını veya silinmesini başlatır.
6. Metadata Saklama	İki temel dosya ile metadata'yı kalıcı tutar: FsImage ve EditLog.
7. Failover Desteği (HA Yapılandırması)	JournalNode ve ZooKeeper ile birlikte yedekleme (standby NameNode) destekler.

HDFS - DataNode

- Data'nın fiziksel olarak depolandığı node'lardır.
- Datanode'lardan namenode'a düzenli aralıklarla heartbeat ve blockreport mesajları iletilir.
- Datanode üzerinde dosyalar block'lar şeklinde depolanır.
- Bu blockların default boyutu 128 MB'tır.
- Replication factor parametresi ile bir bloğun kaç kopyasının oluşturulacağı bilgisi tutulur.
- Belirtilen sayıda block kopyası farklı datanode'lara rack awareness özelliğine uygun şekilde dağıtılır.

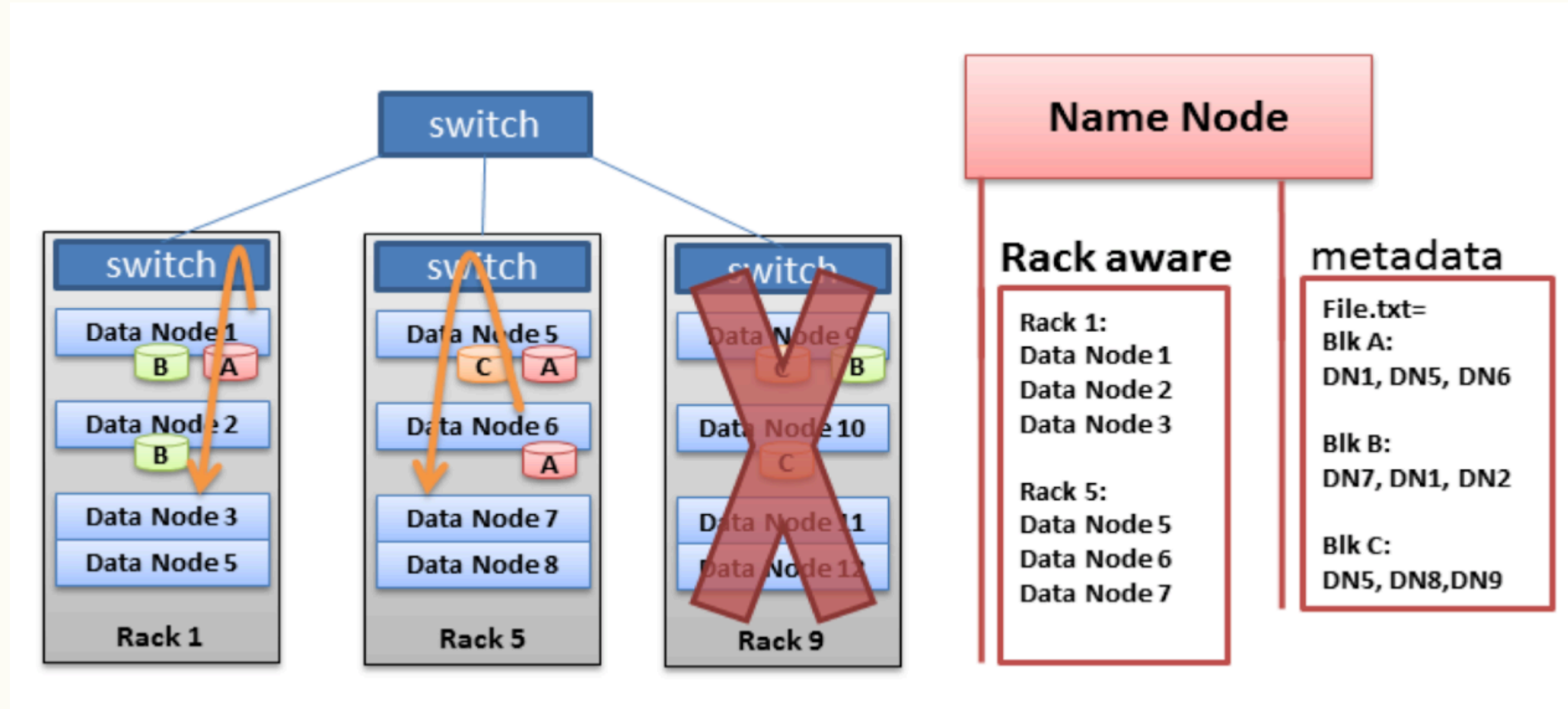


HDFS - DataNode

Rack Awareness

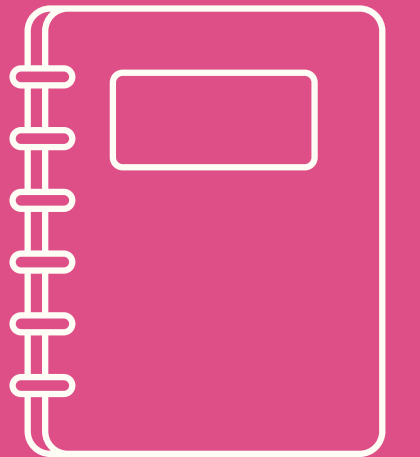
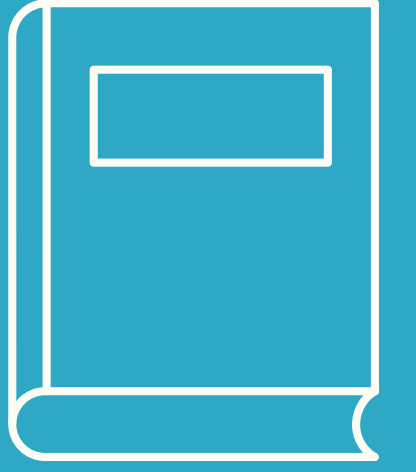
Bir rack, veri merkezindeki fiziksel bir kabin ya da grup sunucudur. Avantajları ise;

- Veri kaybına karşı dayanıklılık
- Ağ verimliliği
- Yük dengeleme



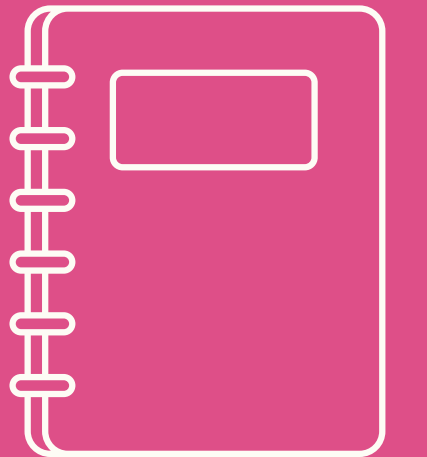
HDFS - FsImage File

- HDFS'in tüm dosya sistemi yapısının (namespace) tam bir anlık görüntüsüdür.
- Dizinler, dosyalar, izinler, sahiplik, zaman damgaları gibi bilgiler içerir.
- Binary formatta, NameNode yeniden başlatıldığında belleğe yüklenir.
- Hangi blokların hangi dosyaya ait olduğu bilgisi yani mapping of block to files bilgisi tutulur.



HDFS - Editlog File

- fsimage yüklendikten sonra yapılan tüm metadata işlemlerinin günlüğüdür.
- Örneğin: dosya oluşturma, silme, yeniden adlandırma gibi her işlem bu log'a yazılır.
- Append-only (sadece eklenir), yani zamanla büyür.
- HDFS metadatası üzerinde yapılan değişiklik kayıtlarının tutulduğu dosyadır. HDFS üzerinde bir dosya oluşturulduğunda hemen editloga bununla ilgili bir kayıt eklenir.



HDFS - Fsimage & Editlog

NameNode Başlatıldığında:

- fsimage dosyası RAM'e yüklenir.
- Ardından, editlog sırasıyla oynatılır (replay edilir).
- Böylece dosya sistemi en güncel haline ulaşır.
- Bu metadata artık RAM üzerinde canlı olarak yönetilir.
- Bu sayede NameNode, her dosya sistemi işlemini RAM'deki yapıyla çok hızlı yönetir.

HDFS - Fsimage & Editlog

Neden Önemlidir?

Risk/Kritiklik	Açıklama
SPOF riski	fsimage ve editlog kaybolursa tüm HDFS yapısı silinir. Veri blokları kalsa bile nereye ait oldukları bilinmez.
Restart süresi uzar	Editlog çok büyürse, NameNode başlatılırken binlerce işlemi tekrar oynatması gerekir (replay latency).
Bellek problemi	Büyük fsimage, RAM yetersizse yüklenemez.

Performans Etkisi

Faktör	Etkisi
Büyük editlog dosyası	Restart süresini uzatır (replay zamanı).
Çok sık fsimage yazmak	Disk I/O yükünü artırabilir.
RAM yetersizliği	fsimage yüklenemez, NameNode çöker.
Yedekleme eksikliği	Arıza durumunda tüm namespace kaybolur.

HDFS -Fsimage & Editlog

Performans İpuçları

- Secondary NameNode (veya Checkpoint Node) Kullan
- Yüksek RAM Kullan
- Yüksek Hızlı Disk Kullan
- Birden Fazla Metadata Yolu Tanımla
- NameNode HA (High Availability) Yapılandır
- Blok yerleştirme ve dosya yönetimi stratejisi belirle



HDFS -Fsimage & Editlog

Fsimage vs Editlog

fsimage	editlog
Tüm dosya sisteminin snapshot'ı	Snapshot sonrası yapılan işlemlerin günlüğü
NameNode başlarken RAM'e yüklenir	Sırasıyla oynatılır (replay)
Periyodik checkpoint ile güncellenir	Checkpoint sonrası sıfırlanır
Kalıcıdır, kritik öneme sahiptir	Büyüdükçe performansı düşürür

HDFS - Fsimage & Editlog

Fsimage vs Editlog File Örnek içerikleri

fsimage dönüştürme (okunabilir hale getirme)

hdfs oiv -i fsimage_00000000000100 -o fsimage.xml -p XML

editlog dönüştürme:

hdfs oev -i edits_00000000000101 -o edits.xml -p XML

-p XML: çıktının XML formatında olmasını sağlar

-i: input (girdi) dosya

-o: output (çıktı) dosya

```
<fsimage>
  <Version>2</Version>
  <NamespaceID>1639328</NamespaceID>
  <files>
    <file>
      <path>/user/veysel/data.csv</path>
      <replication>3</replication>
      <modification_time>1720584912000</modification_time>
      <access_time>1720585203000</access_time>
      <preferred_block_size>134217728</preferred_block_size> <!-- 128 MB -->
      <owner>veysel</owner>
      <group>data_team</group>
      <permission>644</permission>
      <blocks>
        <block>
          <id>blk_107001</id>
          <size>134217728</size>
        </block>
        <block>
          <id>blk_107002</id>
          <size>127506841</size>
        </block>
      </blocks>
    </file>
```

TXID 101: CREATE /user/veysel/data.csv veysel:data_team 644

TXID 102: ADD_BLOCK blk_107001

TXID 103: ADD_BLOCK blk_107002

TXID 104: CLOSE /user/veysel/data.csv

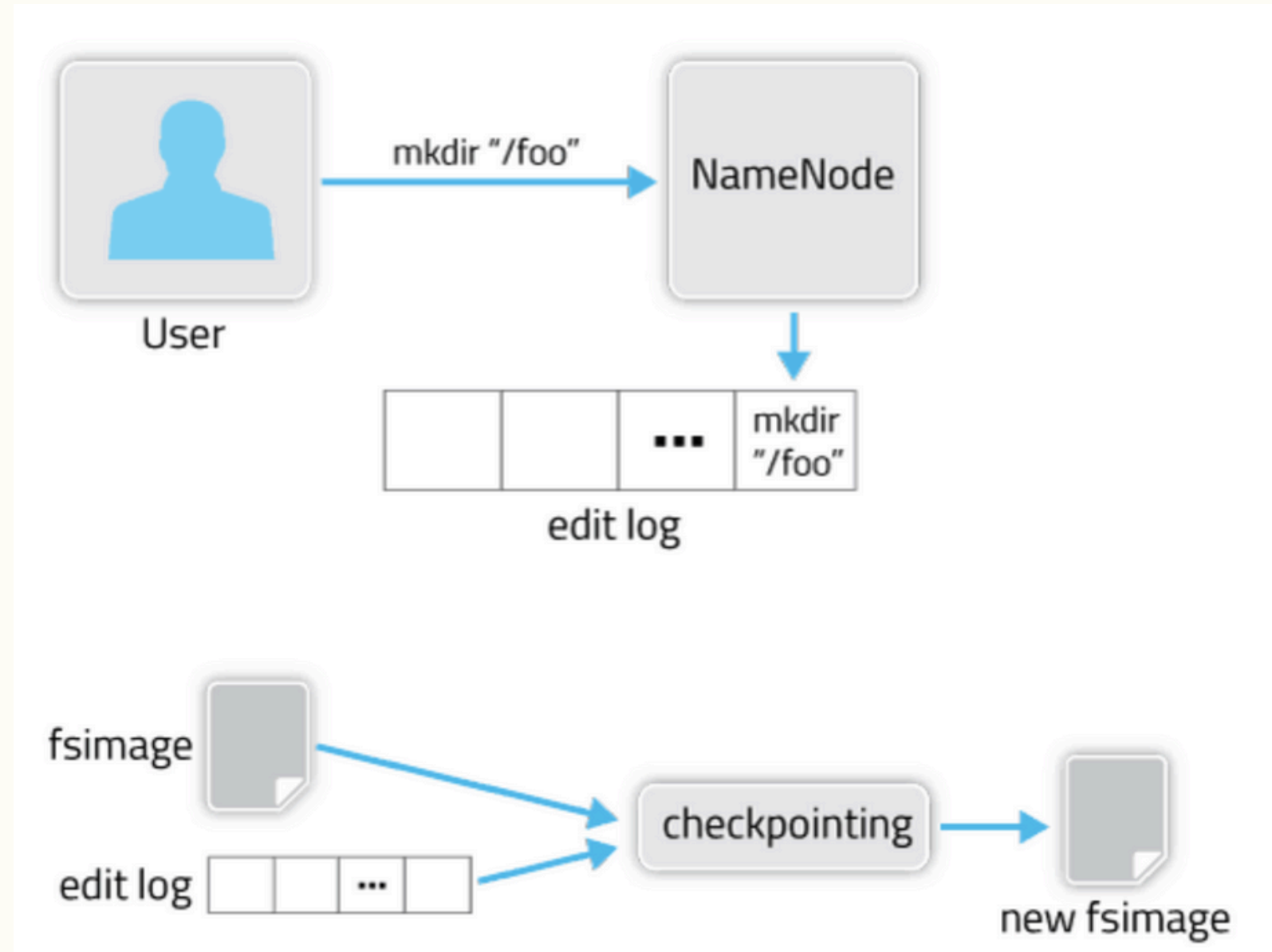
HDFS - VERSION & seen_txid Files

- Fsimage ve editlog dışında sistemde var olan **seen_txid** en son işlenmiş transaction id'sini tutarken,
- **VERSION**, NameNode metadata dizininde bulunan yapılandırma bilgilerini içeren küçük bir dosyadır. İçeriği şu şekildedir;

```
namespaceID=123456789
clusterID=CID-1e89d2a5-2ff9-4a8e-9cce-cd55cda71419
cTime=0
storageType=NAME_NODE
blockpoolID=BP-1234-10.0.0.1-1720590123456
layoutVersion=-63
```

HDFS - Checkpoint İşlemi

- NameNode, düzenli aralıklarla bir checkpoint oluşturur. Checkpoint, FSImage ve Edit Log'un birleşimiyle oluşturulan güncel bir FSImage dosyasıdır.
- Bu işlem, Edit Log'un çok büyük hale gelmesini önler ve sistemin daha hızlı yeniden başlatılmasını sağlar.



HDFS - Checkpoint İşlemi

Checkpoint İşlem Adımları:

1. **fsimage dosyası diskte okunur**
2. **editlog içindeki işlemler sırayla uygulanır** (TXID replay)
3. Elde edilen en güncel metadata:
 - Yeni bir fsimage dosyası olarak diske yazılır
 - Örn: fsimage_0000000010321
4. Eski editlog dosyası silinir ya da sıfırlanır.
5. seen_txid güncellenir.

Checkpoint işlemi iki durumda tetiklenir;

Örnek hdfs-site.xml

```
<property>
  <name>dfs.namenode.checkpoint.period</name>
  <value>3600</value> --saatte bir
</property>

<property>
  <name>dfs.namenode.checkpoint.txns</name>
  <value>1000000</value> -- 1 milyon işlemde bir
</property>
```


HDFS - Checkpoint İşlemi

Diskteki Yeni fsimage Ram'e Yeniden Alınır mı?

Normal Çalışma Sırasında: Hayır.

- Checkpoint sonrası oluşturulan yeni fsimage, sadece diskte güncellenir.
- RAM'deki metadata zaten en güncel haldedir ve tüm client istekleri o RAM'deki canlı yapı üzerinden yönetilir.
- Client → NameNode'a bir istek gönderdiğinde (örneğin dosya oluşturma):
 - Bu istek, RAM'deki metadata yapısı üzerinden işlenir.
 - RAM'deki metadata gerçek zamanlı olarak güncellenir.
 - Aynı zamanda işlem editlog'a yazılır.
 - Checkpoint henüz yapılmadıysa, bu işlem editlog'ta birikir.
 - Yani diskteki fsimage o anda sadece arşiv / yedek konumundadır. RAM'deki metadata ise aktif veri yapısıdır.

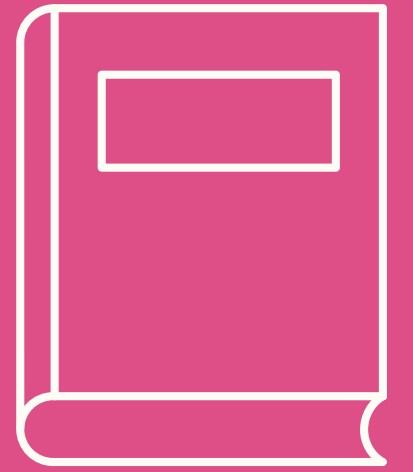
NameNode Restart Olursa: Evet.

- NameNode yeniden başlatıldığında en son fsimage diskten RAM'e yüklenir, ardından editlog uygulanır.

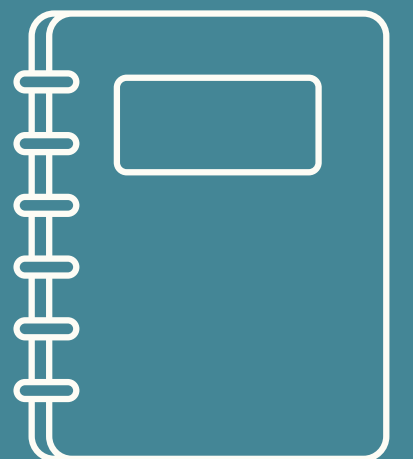
HDFS - Checkpoint İşlemi Özeti

Soru	Cevap
Checkpoint nedir?	fsimage ve editlog birleştirilip yeni fsimage oluşturulmasıdır.
Neden yapılır?	Editlog büyümesin, sistem yavaşlamasın diye.
Kim yapar?	Secondary NameNode, CheckpointNode ya da Standby NameNode (HA).
Ne sıklıkla yapılır?	Zaman ve işlem sayısına göre konfigüre edilir.
Diske yazılan fsimage RAM'e alınır mı?	Sadece restart sırasında alınır. Canlı sistemde değişmez.

HDFS - Secondary NameNode



- Secondary namenode, Namenode'un yedeği değildir.
- Checkpoint işleminin gerçekleştirilmesi sırasında namenode'a yardımcı olur.
- Ana amacı, NameNode'un üzerindeki yükü azaltmak ve metadata dosyalarının (fsimage ve editlog) yönetimini optimize etmektir
- Sadece fsimage ve editlog dosyalarının birleştirilmesi (checkpointing) işlemini yapar.
- NameNode ile belirli aralıklarla iletişime geçerek bu dosyaları alır, işler ve sonucu tekrar NameNode'a gönderir.



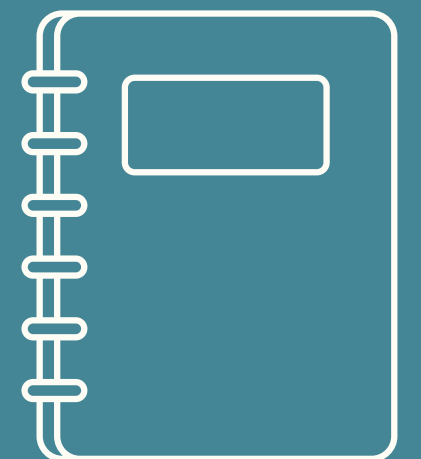
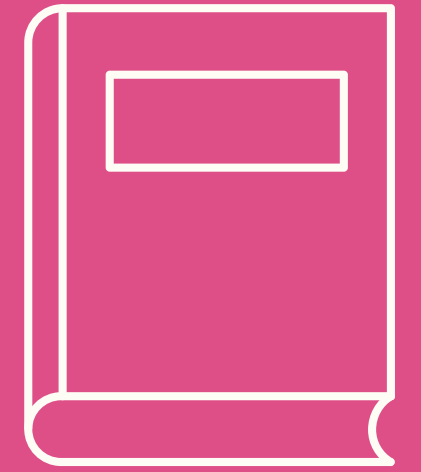
Secondary NameNode, Namenode'un **YEDEĞİ DEĞİLDİR** :)

HDFS - Secondary NN & Checkpoint

- HA olmayan bir clusterda, checkpoint işlemi SecondaryNameNode tarafından yapılır. Ancak bu durumda, edit log dosyaları paylaşımında kullanılan ortak bir dizin bulunmadığından, SecondaryNameNode ek adımlar atmak zorundadır:
 - Edit Log Dosyalarını alma.
 - Güncelleme İşlemleri.
 - FSImage ve Edit Log'u Birleştirme
- Bu ek adımlardan sonra, SecondaryNameNode standart checkpointing adımlarını takip ederek, dosya sisteminin güncel ve güvenilir bir görüntüsünü sağlar.

HDFS - Standby NameNode

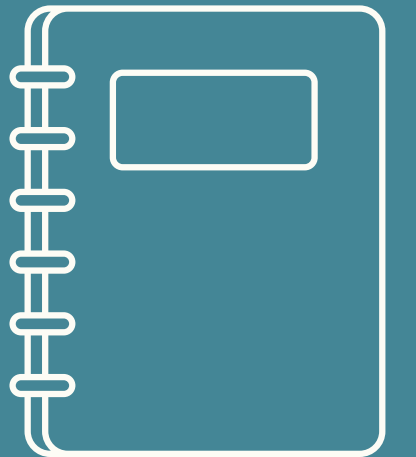
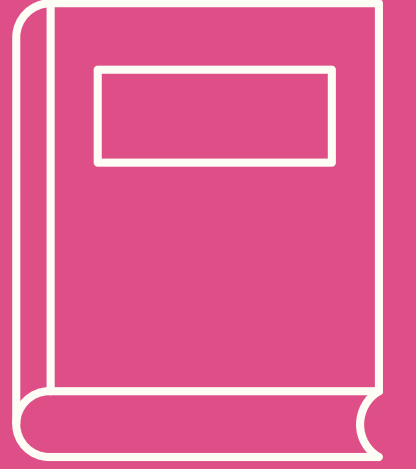
- **Standby Namenode**
- **SBNN:** Active NameNode'un bir kopyasıdır.
- HDFS metadata'sını eş zamanlı (senkron) olarak takip eder.
- Active NameNode çökerse otomatik olarak devreye girer ve HDFS yönetimini devralır → bu işleme failover denir.



Stanby NameNode, Namenode'un **YEDEĞİDİR :)**

HDFS - Journal Node

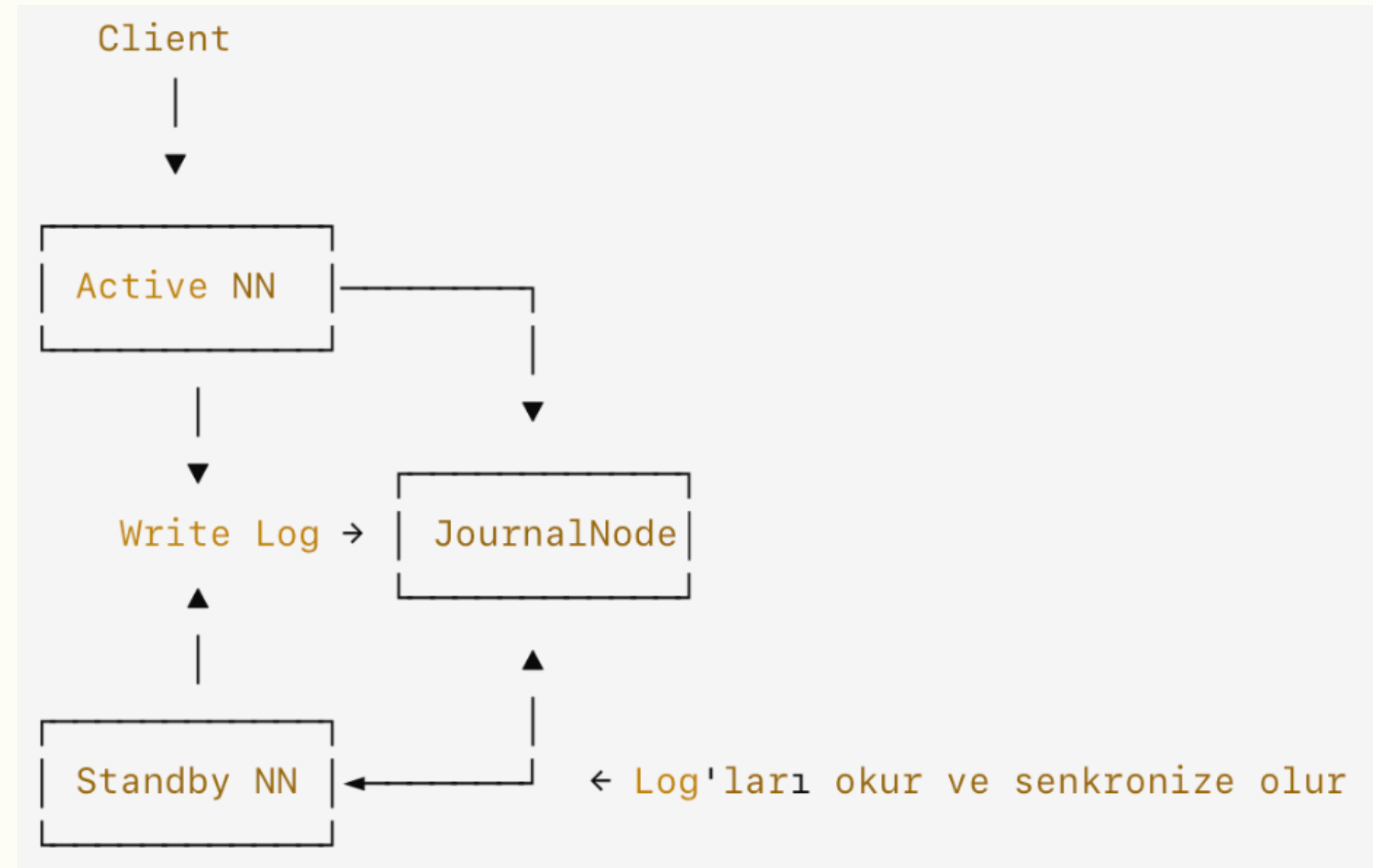
- JN: Active ve Standby NameNode arasında metadata deęişikliklerini paylaşan bir g nl ę  (editlog) tutar.
- Active NameNode, yaptığı her deęişiklięi birden fazla JournalNode'a yazar.
- Standby NameNode bu deęişiklikleri okur ve kendi belleęini senkronize eder.
- **Journal Node'lar ile HDFS NameNode'a high availability  zellięi kazandırılır.** Ve veri tutarlılıęında  nemli rol oynar. Temel g revi editlog dosyalarını saklayarak aktif ve pasif namenode'lar arasında senkronizasyonu saęlamaktır. JournalNode'lar edit log dosyaları i in shared storage gibi davranır. Ve NN tarafından oluřturulan editlogları HA prensiplerine uygun olarak saklar. Bu sayede NN i in failover iřlemi gerektięinde kolaylıkla standby NN devreye alınır.



HDFS - Journal Node

Client, Active NameNode'a dosya oluşturma isteği gönderir.

- Active NameNode bu işlemi:
- Kendi RAM'indeki metadata'ya uygular
- En az 3 JournalNode'a aynı işlemi yazar (editlog entry)
- JournalNode'lar bu işlemi diske kaydeder.
- Standby NameNode, bu JournalNode'ları izler ve:
- Aynı işlemleri sırasıyla okuyarak kendi belleğine uygular.
- Böylece Active ile senkronize kalır.



HDFS - Failover işleyişi

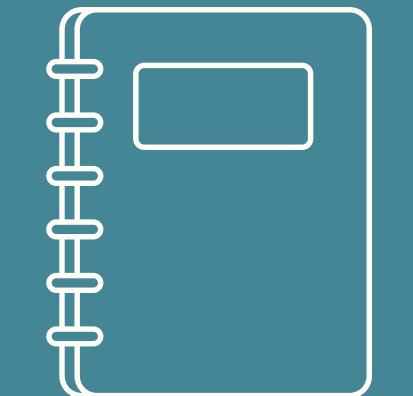
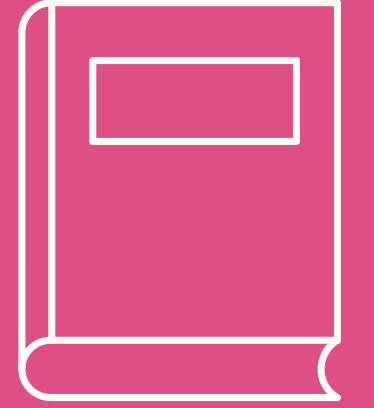
- ZooKeeper üzerinden bir seçim yapılır (Active kim olacak?).
- Standby NameNode → Active olur.
- JournalNode'daki en güncel işlemleri okuyup, yönetimi devralır.
- Client istekleri artık yeni Active NameNode üzerinden akar.

Zookeeper

- Merkezi bir koordinasyon servsidir.
- Dağıtık sistem bileşenlerinin:
- Birbirlerini keşfetmesini
- Lider seçimi yapmasını
- Durum bilgisi paylaşmasını
- Kilit mekanizmaları kurmasını sağlar.

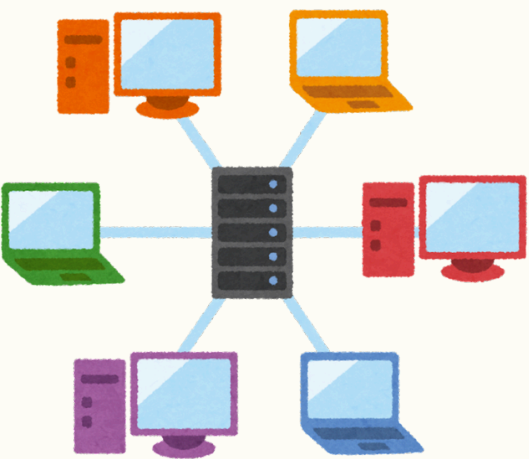
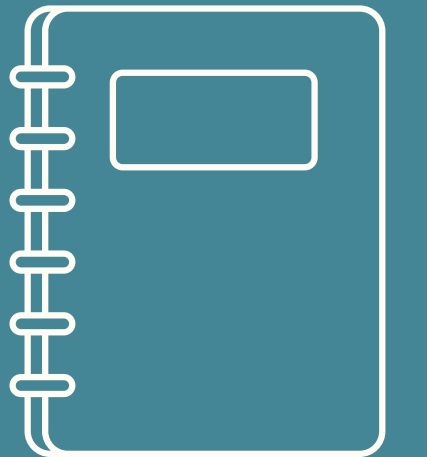
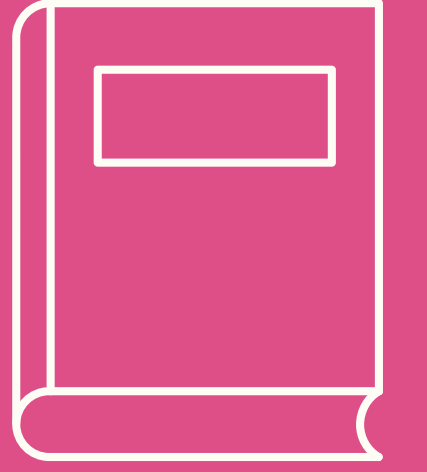
HDFS HA Mimaride ZooKeeper'ın Rolü

- Active / Standby NameNode'lar arasında koordinasyon sağlar.
- ZooKeeper, hangi NameNode'un **aktif** olacağına karar verir.
- Arıza durumunda failover işlemini tetikler.

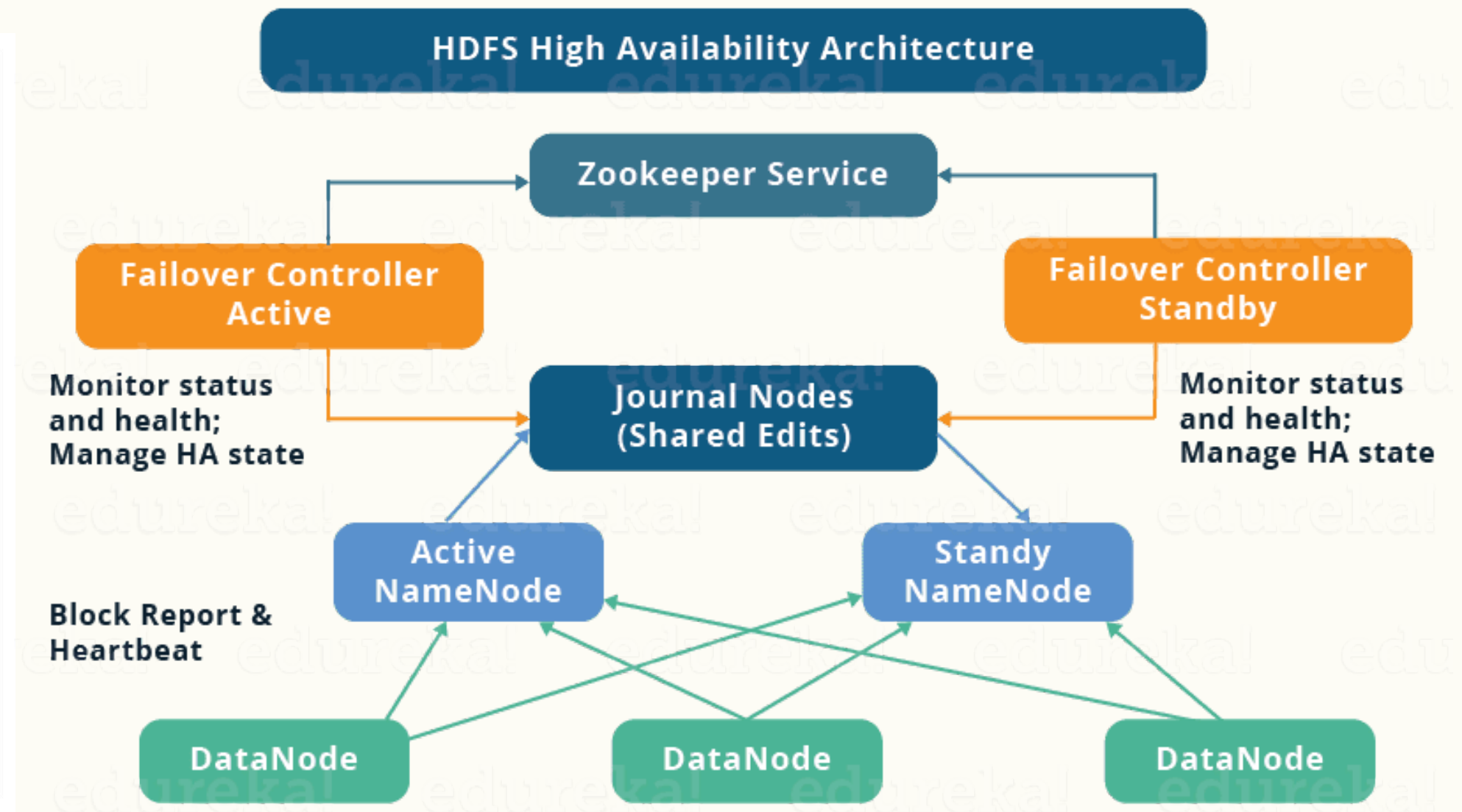
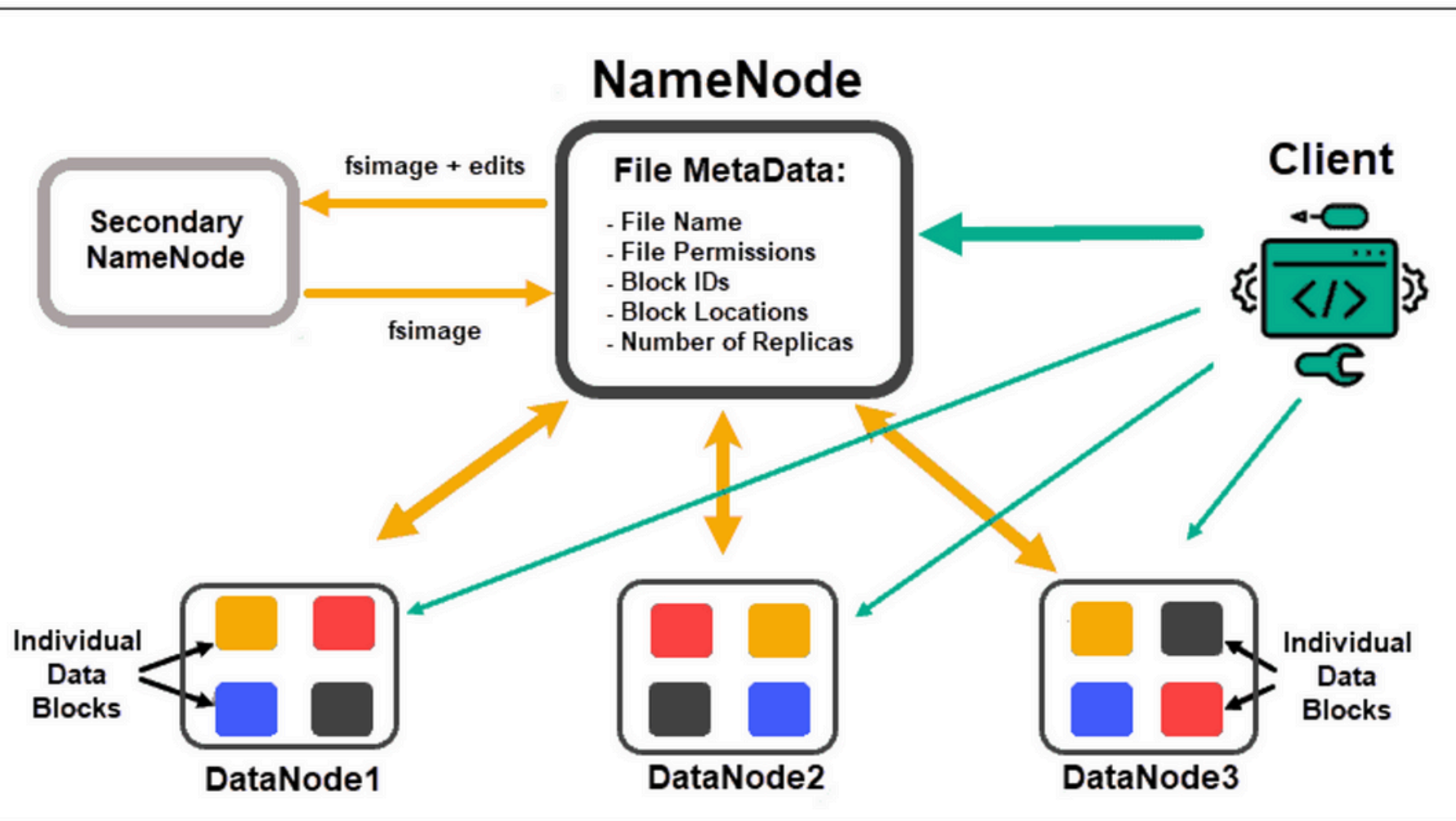


HDFS - Edge Node

- Edge node'lar için HDFS'in dış dünya ile bağlantısını, client erişimini sağlayan nodelardır.
- Hadoop cluster'a erişim noktası gibi çalışır.
- Kullanıcılar bu node üzerinden:
 - HDFS'e veri yükleyebilir veya sorgulayabilir
 - YARN üzerinden Spark / MapReduce job'ları gönderebilir
 - Hive, HBase gibi servislere bağlanabilir
 - Cluster içindeki diğer node'lara göre daha az kritik, ama dış dünyaya en açık node'dur.



HDFS - Architectures



HDFS - Replication

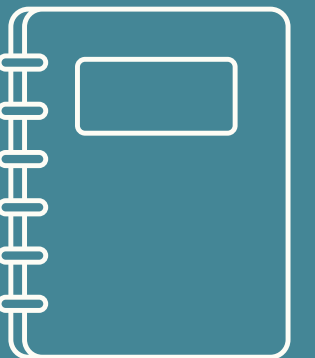
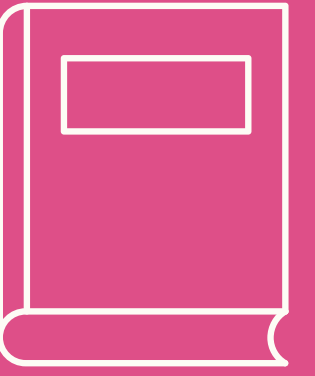
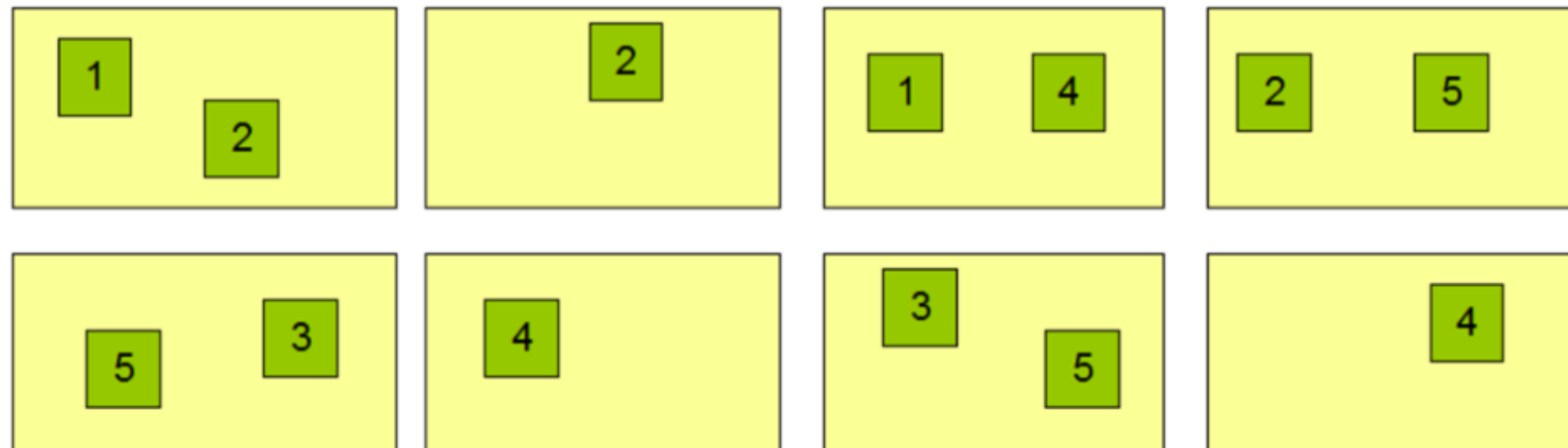
- HDFS, dosyaları bir blok dizisi şeklinde depolar.
- Tutulan dosyaların hata durumundan etkilenmemesi için, blok bazında gerçekleştirdiği replikasyon özelliği sayesinde dosyaların kopyalarını da depolar.
- Dosya bloklarının boyutu, **Block Size** parametresi ile belirlenir.
- Her bloğun kaç kopyasının tutulacağı bilgisi ise **Replication Factor** parametresi ile belirlenir.
- Bir dosyanın son bloğu hariç, bütün blokları aynı boyuttadır.

NameNode (Dosya Adı, Replica Sayısı, Blok ID)

/buyukveri/data/part-0,r:2,{1,3}

/buyukveri/data/part-1,r:3,{2,4,5}

DataNodes



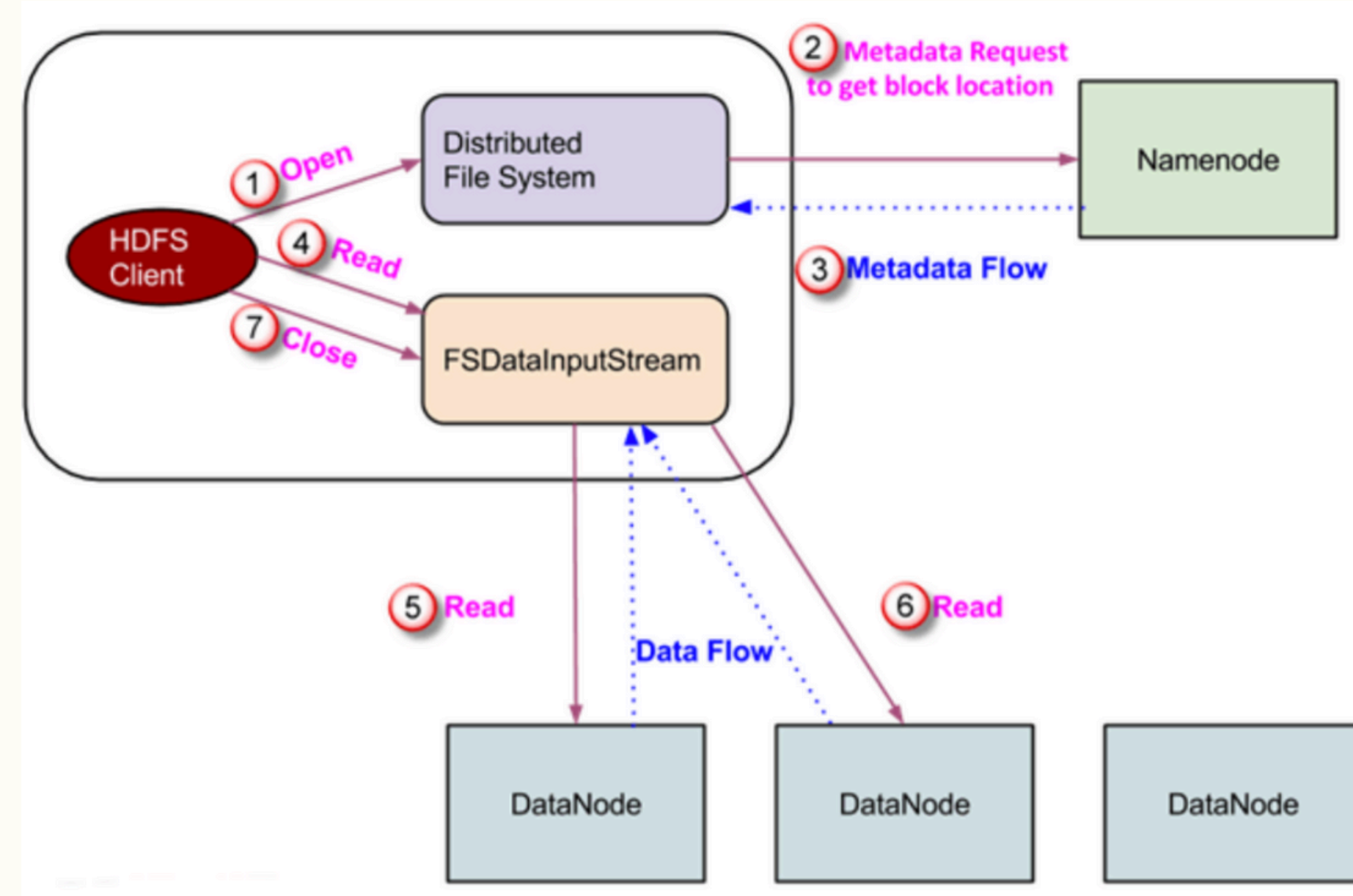
HDFS - Trash

HDFS Trash

- Bir dosya silinmek istediği zaman, o dosya HDFS üzerinden hemen kaldırılmaz.
- Silinen dosyaların tutulduğu ve trash dizini olarak adlandırılan dizince tutulmaya devam edilir.
- Her kullanıcının kendisine ait bir trash dizini bulunur.
- **(/user/<username>/.Trash).**

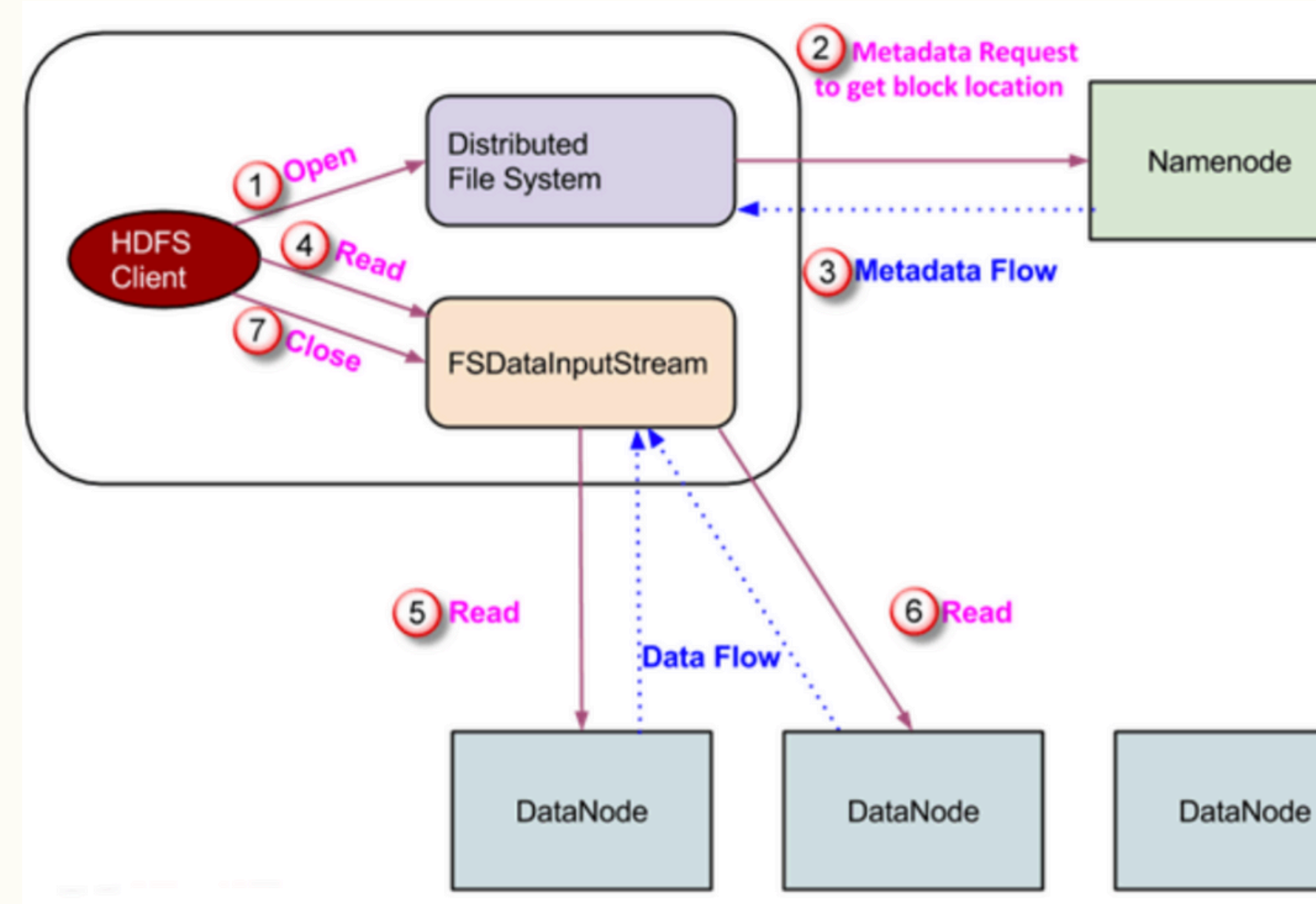
HDFS - READ OPERATION

1. Kullanıcı tarafından Distributed File System üzerinde open() fonksiyonu çalıştırılarak bir read request başlatılır.
2. Bu adımda Client tarafından read request için başlatılan obje NameNode'a bağlanılır ve metadata bilgisini alarak ilgili blokların Cluster'daki yerini bulur.
3. Datanode'ların yerini içeren metadata bilgisi Client'a gönderilir
4. DataNode'ların yeri alındıktan sonra kullanıcıya bir de FSDataInputStream adında bir obje döndürülür ve bu obje aracılığı ile Client DataNode'lar ile iletişime geçer. Clienttan gelen Read komutu ile DataNode ile bağlantı sağlanır ve dosyanın ilk bloğu okunur.
5. Okuma işlemi son dosya bloğuna kadar devam eder.
6. İlgili DataNode'daki son dosya bloğu okunduktan sonra FSDataInputStream bağlantıyı kapatır ve diğer blokları tutar DataNode'lara bağlanarak okumayı gerçekleştirir.
7. Okuma işlemi tamamlandığında client tarafından bir close() komutu FSDataInputStream objesine gönderilir ve işlem tamamlanır.



HDFS - WRITE OPERATION

1. Client tarafından Create() fonksiyonu ile HDFS üzerinde yeni bir dosya yaratma süreci başlatılır.
2. Bu adımda Yaratılan DFS objesi NameNode'a bağlanır. NameNode aracılığı ile client'ın dosya yaratma yetkisinin olup olmadığı ve sistemde yaratılmak istenen isimde bir dosya olup olmadığı kontrol edilir. Eğer bu şartlar sağlanmaz ise kullanıcıya IOException hatası döndürülür. Eğer Bu şartlar sağlanır ise NameNode üzerinde Namespace güncellenir ve yeni dosya oluşturma işlemi başlatılır
3. Yeni dosya için namespace kaydı oluşturulduktan sonra; Client'a FSDataOutputStream tipinde bir obje döndürülür. DataNode'a yazma işlemi için bu obje kullanılır. Client tarafından yazma işlemi bu adımda tetiklenir.
4. FSDataOutputStream objesi içinde yer alan DFSOutputStream objesi Clienttan gelen verileri paketler ve DataQueue'da sıraya sokarak yazmaya hazır hale getirir.
5. DataStreamer tarafından NameNode'dan yazılacak blokların kaç replikasının tutulacağı bilgisi alınır.
6. Replika bilgisine göre DataNode'lar arasında pipeline oluşturulur.
7. DataStreamer tarafından ilk Datanode'a kuyrukta bekleyen paketlenmiş veri aktarılmaya başlanır.
8. Daha sonra Datanode tarafından yazılan aynı paket pipeline'a dahil olan diğer DataNode'lara yazılmak üzere iletilir.
9. Dataqueue dışında yine DFS tarafından yönetilen bir de ACK queue oluşturulur. Veri bloklarının başarılı şekilde yazılıp yazılmadığı bilgisi bu kuyrukta tutulur.
10. Bir paket için pipelineda yer alan bütün datanode'lardan ACK bilgisi alındığı zaman o paket ACK kuyruğundan çıkarılır. Eğer ACK bilgisi alınamaz ise kullanıcı tarafından bu paket yazılmak üzere tekrar gönderilir.
11. Paketlerin yazılma işlemi bitirildikten sonra Client tarafından Close method gönderilir ve yazma işleminin tamamlandığı bilgisi DFS'e verilir.
12. En son paket için ACK bilgisi alındığında NameNode ile iletişime geçilerek yazma işleminin tamamlandığı bilgisi NameNode'a verilir.



HDFS - Small Files Problem

Blocks size 256MB						
HDFS Blocks	1	2	3			
A	File 1	File 16	File 31		Editlog'a 30 farklı kayıt	Block boyutu / Yazılan data
B	File 2	File 17	File 31		TXID1: File1 created, BlockID A1	256/10
C	File 3	File 18			TXID2: File2 created, BlockID B1	256/10
D	File 4	File 19			TXID3: File3 created, BlockID C1	256/10
E	File 5	File 20			TXID4: File4 created, BlockID D1	256/10
F	File 6	File 21			TXID5: File5 created, BlockID E1	256/10
G	File 7	File 22				
H	File 8	File 23				
I	File 9	File 24				
J	File 10	File 25				
K	File 11	File 26				
L	File 12	File 27			TXID30: File30 created, BlockID O2	256/10
M	File 13	File 28			TXID31: File31 created, BlockID A3,B3	256/256 , 256/44
N	File 14	File 29				
O	File 15	File 30				

10 MB'lık 30 dosya ve 300 MB'lık 1 dosya 'nın hdfs'te depolanması sırasında;

- File1-30 , bunlar 256 MB'lık block'lara her bloğa tek dosya olacak şekilde yazılır, kapladıkları alan da her blockta 10MB olur ve 246MB'lık boş alan kalır bu blocklarda.
- File 31 ise 300MB'lık bir dosya ve 256 Mb'ı bir bloğa yazılırken kalan 44 MB'ı ayrı bir bloğa yazılır.

Dolayısı ile aynı boyutta-300MB veri depolanırken;

- 1.senaryoda 30 block kullanılır ve 30 adet editlog kaydı oluşur. Daha fazla metadata operasyonu daha düşük performans demektir. Editlog kayıtlarının fsimage dosyasının büyümesi açılış sırasında da yavaşlığa sebep olur.
- 2.senaryoda ise 2 block kullanıldı. 1 Editlog kaydı oluştu. Ve görüldüğü gibi **1.senaryoya göre 30 kat daha az metadata operasyonu gerçekleşti.**

Neler Öğrendik

Hadoop Ekosistemi

HDFS nedir?

HDFS'de Node Roller(NN, SBNN, Secondary NN, Datanode, JN, Edge Node) ve Görevleri.

HDFS HA ve Single NN mimarileri.

HDFS checkpoint işlemi

HDFS Editlog, Fsimage dosyaları

HDFS Metadata Yönetimi

Fsimage ve Editlog Performans ipuçları

Small Files Problem

HDFS Replication

Failover işlemi



Sorular?

